

Programming (Python) for Deep Learning

ECC504

Start with basic questions...

- When to use deep learning
- Why Python

When to use Deep Learning

- When the **data is large-scale** (images, audio, text, sensor streams).
- When the **task is complex** (vision, speech, NLP, generative AI) and traditional ML fails.
- When you need **automatic feature extraction** (no handcrafted features).

Why Python wins for deep learning



- Huge ecosystem (Core Frameworks, High-Level Libraries, pre-trained models)
- Community support
- Easy integration with C/CUDA for speed
- Vast community/tutorials. Most state-of-the-art models and papers release Python code first.
- Easy syntax

Python gives you the fastest path from idea → experiment

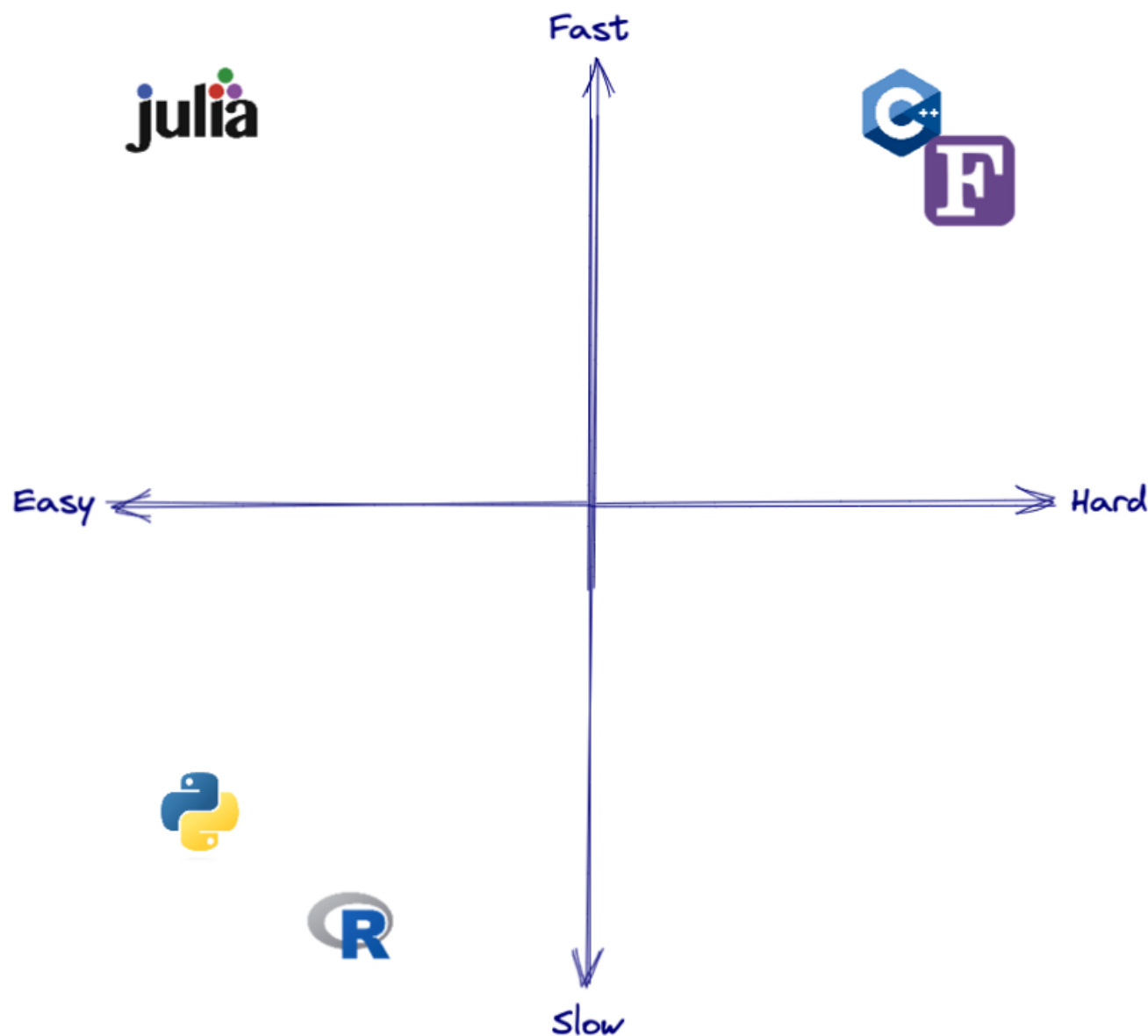
Alternatives: R, Julia, C++, Java, MATLAB , CUDA, etc.

If you want modern alternative → Julia (for research).



- Julia is an open-source programming language introduced in 2012 by Alan Edelman et. al.
- It is created for high-performance scientific computing with high-level code.

- Python = Present & Near Future of DL (ecosystem, community, pretrained assets).
- Julia = Possible Long-Term Future for HPC + scientific AI, if ecosystem grows.



Reference book

DEEP LEARNING with Python

François Chollet

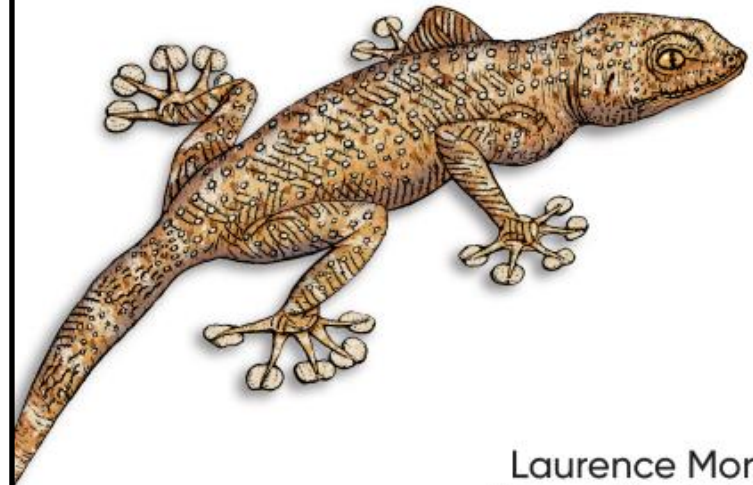
 MANNING



O'REILLY®

AI and Machine Learning for Coders

A Programmer's Guide to Artificial Intelligence



Laurence Moroney
Foreword by Andrew Ng

What is a Deep Learning Framework?

- Provides a high-level API to design, train, and deploy models.
- Handles numerical/ algebraic computation (e.g. differentiation) and optimization.
- Allows deployment on different types of H/W Platforms

High-level API

Just a few lines: model defined, compiled, and trained.

High-level API:

for beginners & rapid prototyping.

python

 Copy code

```
from tensorflow import keras
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=10)
```

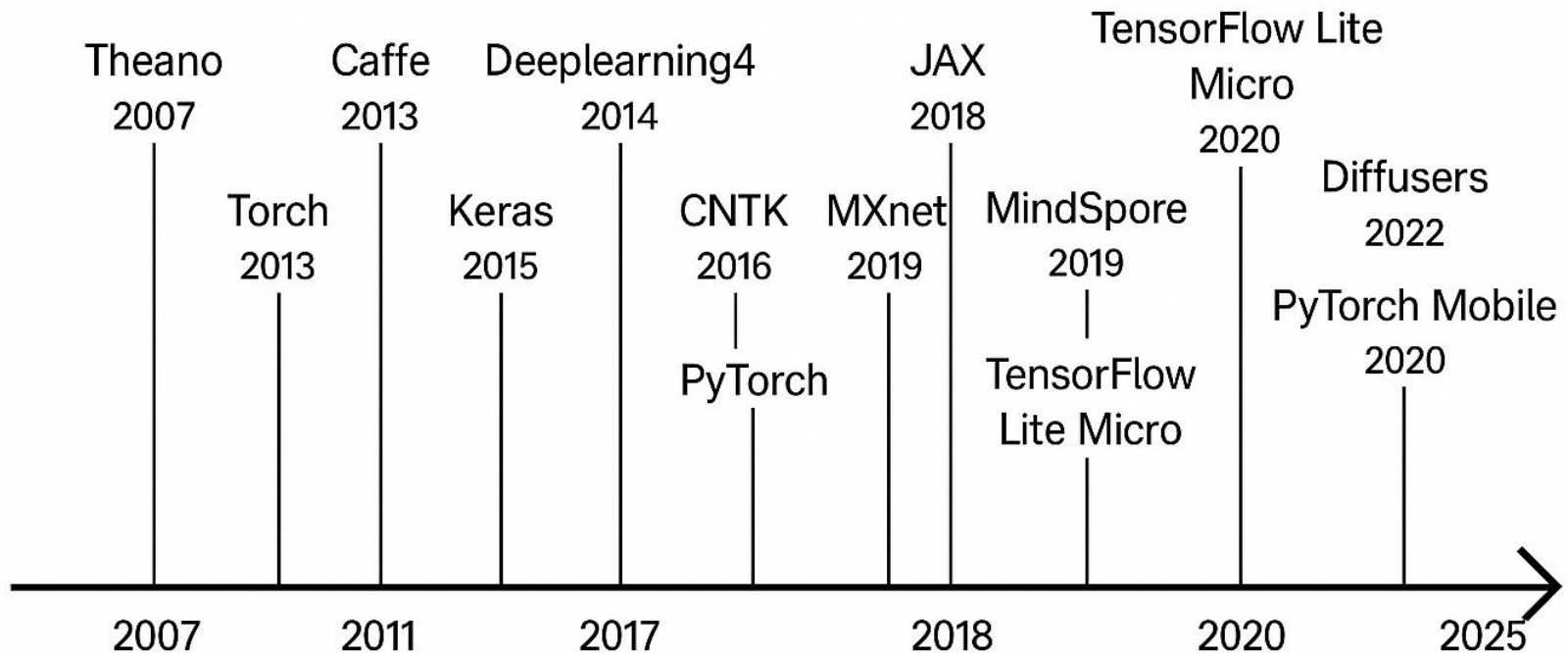
Low-level API (manual):

You would have to write the forward pass, loss calculation, backpropagation, and optimizer update manually with tensor operations. **Much more flexible, but harder** and verbose.

Evolution of Frameworks

- Theano (2008, pioneering but discontinued).
- Caffe (limited flexibility).
- TensorFlow (2015) -- Google
- PyTorch (2016) – Meta/FB
- JAX (2018) -- Google

Timeline of Deep Learning Frameworks



Why Deep Learning Frameworks
Started to Boom from 2007-2008

1. Breakthrough Research (2006–2007)

- 2006 → Geoffrey Hinton's paper on *Deep Belief Networks* showed that deep neural networks could be trained effectively using **layer-wise pretraining**.
- This renewed interest in *deep* neural networks after the "AI winter."



Geoffrey Hinton

Emeritus Prof. Computer Science, University of Toronto

John J. Hopfield

Geoffrey E. Hinton

"for foundational discoveries and inventions
that enable machine learning
with artificial neural networks"

THE ROYAL SWEDISH ACADEMY OF SCIENCES



2006

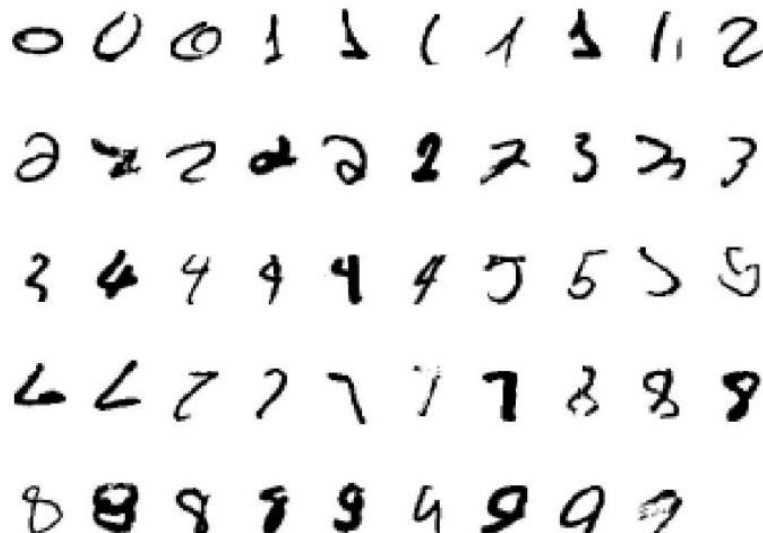
A fast learning algorithm for deep belief nets *

Geoffrey E. Hinton and Simon Osindero

Department of Computer Science University of Toronto
10 Kings College Road
Toronto, Canada M5S 3G4

Yee-Whye Teh

Department of Computer Science
National University of Singapore
3 Science Drive 3, Singapore, 117543
tehyw@comp.nus.edu.sg



MNIST dataset

Deep learning frameworks experienced significant growth from 2007 onwards, thanks to the availability of GPUs (CUDA), big data, and Theano, which made it possible to train deep models practically, while Hinton's research demonstrated their effectiveness.

Why GPU is Required for Deep Learning

....needs GPUs because of massive parallel math

Why CPU is not enough

- CPU = few powerful cores, optimized for sequential tasks.
- GPU = thousands of smaller cores, optimized for **parallel math**.

Example

- Training ResNet-50 on ImageNet:
 - CPU only → weeks.
 - GPU (e.g., NVIDIA V100) → hours.
- That's why practical DL became possible only after GPU acceleration.

◆ 1. What is GPU support in Deep Learning?

- GPU (Graphics Processing Unit) → originally for graphics, but has thousands of parallel cores.
- Deep Learning needs massive matrix multiplications & tensor ops → GPUs accelerate these far better than CPUs.
- GPU support in DL means frameworks (PyTorch, TensorFlow, JAX, etc.) can automatically use GPUs for training/inference.
- Without GPU → training a large CNN may take **days**; with GPU → **hours/minutes**.

Why GPU is Required for Deep Learning

1. Matrix & Tensor Operations

- Core of DL = **matrix multiplications, convolutions, vector ops.**
- Example: multiplying a 1000×1000 matrix by another can be split into **millions of independent multiplications.**
- Each multiplication does not depend on others → can run in parallel.

2. Neurons/Activations

- In a neural layer, each neuron's activation is computed using the **same formula** but different weights/inputs.
- All neurons in the same layer can be calculated **independently** → ideal for parallelism.

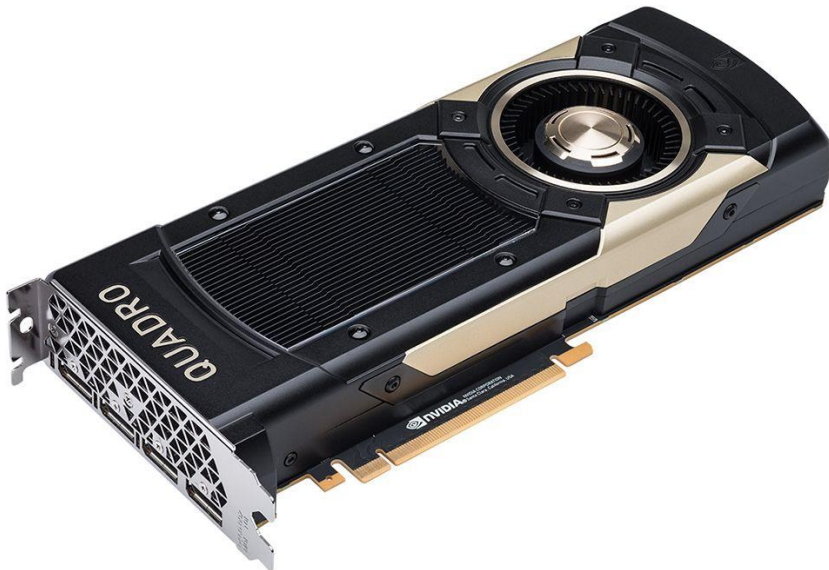
3. Mini-Batch Training

- Training uses **mini-batches** of data.
- Each sample's forward + backward pass can be computed separately.
- Gradients from all samples are then aggregated → again **parallel-friendly.**

GPUs could accelerate **linear algebra** (the core of DL).



**NVIDIA in 2007 launched CUDA —
making GPUs programmable for
general-purpose computing**



**Since then, NVIDIA has dominated
AI research/industry**

CUDA (Compute Unified Device Architecture) is NVIDIA's parallel computing platform.

Around 2007, NVIDIA's CUDA was released.

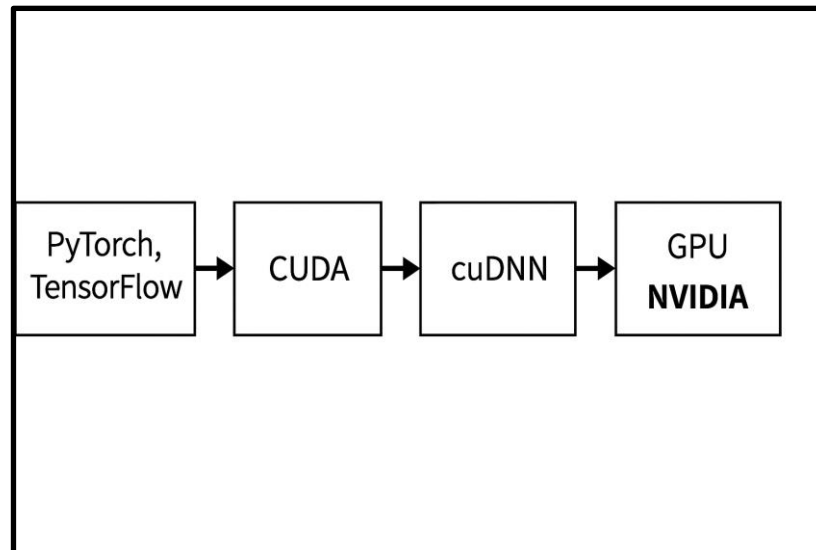
- Think of it as a **bridge between DL frameworks and the GPU hardware.**
- Without CUDA, a framework can't talk to the GPU.

The CUDA Toolkit = software package provided by NVIDIA

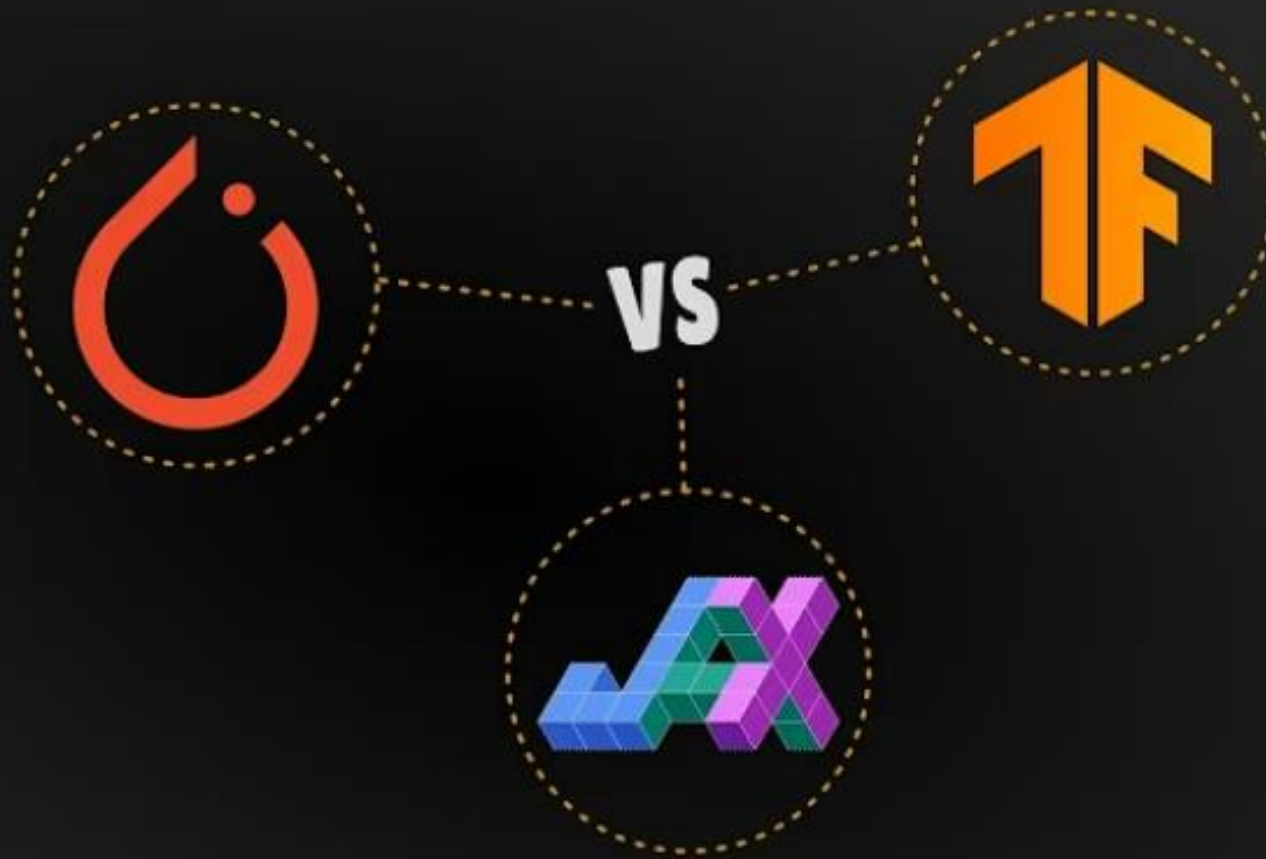
- **cuDNN (CUDA Deep Neural Network library)** → part of CUDA toolkit.
- Highly optimized kernels for DL (convolutions, RNNs, pooling, normalization).
- Frameworks (PyTorch/TF) call cuDNN functions internally → you don't need to code low-level GPU kernels yourself.

Simple Intuition...

- **GPU** = super-fast kitchen with many chefs.
- **CUDA** = language that lets you give orders to the chefs.
- **CUDA Toolkit + cuDNN** = recipe books and tools for standard dishes (matrix multiplications, convolutions).
- **PyTorch & TensorFlow** = friendly waiters who take your high-level order ("train CNN") and talk to the chefs in CUDA language.



PyTorch vs TensorFlow vs JAX



Workflow of Model Development

- Import libraries & dataset.
- Define model architecture.
- Compile with optimizer & loss.
- Train & validate.
- Evaluate & save model.
- Deploy (edge/cloud).

Steps to know before coding

- How to create programming setup in IDE
- Cloud GPU or own server/PC

Environment Setup Commands

1. Create a new Conda environment with Python 3.6

```
conda create --name sumanEnv python=3.6
```

2. Activate the environment

```
conda activate sumanEnv
```


3. Install required Python packages (using pip)

```
pip install pandas
```

```
pip install xlrd
```

```
pip install seaborn
```

```
pip install scikit-learn
```

```
pip install librosa
```

```
pip install tensorflow
```

```
pip install matplotlib
```

```
pip install ffmpeg
```

```
pip install glob2
```

```
pip install ipykernel
```

4. Install Jupyter (using conda for compatibility)

```
conda install jupyter
```

TensorFlow/Keras

- TensorFlow: powerful, graph-based execution.
- Keras: user-friendly API (Sequential, Functional).
- Ecosystem: TFX, TensorFlow Lite, TensorBoard.
- Widely used in industry & production pipelines.

PyTorch

- Dynamic computation graph (define-by-run).
- Simple, Pythonic, widely adopted in research.
- TorchVision, TorchAudio, TorchText libraries.
- PyTorch Lightning for structured training.

JAX

- NumPy-like syntax with GPU/TPU acceleration.
- Automatic differentiation via `grad()`.
- Popular in Google DeepMind, reinforcement learning.
- Libraries: Flax, Haiku, Optax.

Building Blocks

- Tensors: n-dimensional arrays.
- Layers: Dense, Conv2D, RNN, Transformer blocks.
- Loss Functions: Cross Entropy, MSE.
- Optimizers: Adam, SGD.
- Training loop: forward → loss → backward → update.

Workflow of Model Development

- Import libraries & dataset.
- Define model architecture.
- Compile with optimizer & loss.
- Train & validate.
- Evaluate & save model.
- Deploy (edge/cloud).

Advanced Features

- Transfer learning (pretrained models).
- Distributed training (multi-GPU/TPU).
- Model compression: quantization, pruning.
- ONNX for interoperability.
- TinyML: deployment on microcontrollers.

Trends & Future Directions

- Generative AI (Transformers, Diffusion models).
- Agentic AI frameworks (LangChain, LlamaIndex).
- JAX dominance in research.
- Hardware-aware model design (TinyML, NAS).

Summary

- Frameworks simplify deep learning development.
- TensorFlow → Industry, PyTorch → Research, JAX → Future research.
- Key skills: tensor operations, model building, training loop.
- Deployment and scaling are as important as training.